Lab 5: Out-of-Order Lab

In this lab, you will use the SUSTemu RISC-V simulator to explore how an out-of-order (OOO) processor extracts instruction-level parallelism (ILP) and hides memory latency. By the end of this lab, you will be able to:

1. Understand what limits OOO IPC in practice
2. Predict instruction issue cycles using the Tomasulo timing model
3. Measure how OOO hides memory latency through independent loads

Default OOO configuration (`include/cpu/exec_cfg.h` and `include/cpu/ooo.h`):

- Issue width: **2** instructions/cycle · INT units: **2** · MUL units: **1** · LSU units: **2**
- `LAT_INT_ADD = 1` · `LAT_INT_MUL = 3` · `LAT_L1_HIT = 1` · `LAT_L2_HIT = 8`
- ROB: **32** entries · RS: **16** entries

Step 1 — Instruction-Level Parallelism

Environment Setup

Question 1: Clone the repository and build the simulator:

```
git clone git@github.com:Compass-All/SUSTemu.git
cd SUSTemu
make clean && make
```

Run `whoami` in the same terminal after a successful build.

Submit: a screenshot showing both the build output ending with `over` and the `whoami` result.

Each benchmark directory provides two make targets:

```
make run-inorder   # 5-stage in-order pipeline
make run-ooo       # out-of-order engine (Tomasulo + ROB)
```

> **Note:** after modifying any file under `include/`, always run `make clean && make`.

Background

`ooo_lab/Q1/ilp/` contains two kernels with the same number of iterations:

- `kernel_serial` — pointer-chasing: `sum += aa[sum & (NELEM-1)]`; the load address of each iteration depends on the accumulated `sum` from the previous iteration — no look-ahead possible
- `kernel_parallel` — two independent accumulators: `sa += aa[i]; sb += bb[i]`; the two chains share no data dependency and can advance simultaneously

The working set (8 KB) fits entirely in L1D, so all loads are L1 hits.

The compiled inner loops (RISC-V, -O2) are shown below for reference:

kernel_serial (7 instructions/iteration):

```
and   a5, a0, 511      ; idx = sum & (NELEM-1)  — depends on sum (a0)
sll   a5, a5, 3        ; idx <<= 3              — depends on and result
add   a5, a3, a5       ; addr = base + idx      — depends on sll result
ld    a5, 0(a5)        ; load aa[idx]           — depends on computed addr
addw  a4, a4, -1       ; i--                    — independent
add   a0, a0, a5       ; sum += loaded          — depends on ld result
bnez  a4, <loop>
```

kernel_parallel (7 instructions/iteration):

```
ld    a1, 0(a5)        ; load aa[i]       — a5 is aa pointer (updated by
prev iter)
ld    a2, 0(a4)        ; load bb[i]       — a4 is bb pointer (updated by
prev iter)
add   a5, a5, 8        ; aa pointer++     — loop-carried via a5
add   a0, a0, a1       ; sa += aa[i]      — depends on ld result AND prev
sa
add   a3, a3, a2       ; sb += bb[i]      — depends on ld result AND prev
sb
add   a4, a4, 8        ; bb pointer++     — loop-carried via a4
bne   a6, a5, <loop>   ; loop branch      — depends on a5 (this iter)
```

Run both kernels in both modes:

```
cd ooo_lab/Q1/ilp
make run-inorder
make run-ooo
```

	In-Order	OOO
serial		
parallel		

Question 2: Both kernels have the same instruction count per iteration (7 instructions). Using the dependency chains visible in the assembly above, explain why `kernel_parallel` achieves significantly higher OOO IPC than `kernel_serial`. Identify the critical path for each kernel and explain what limits the OOO scheduler in each case.

Submit: completed table and written answer to Q2.

Step 2 — Tomasulo Scheduling

Background

`ooo_lab/Q2/tomasulo/` contains two benchmarks that differ only in dependency structure:

- `bench_diamond`: `b = a+1; c = a*a; d = b+c;` — B and C **both depend on A** (parallel fan-out)
- `bench_chain`: `b = a+1; c = b*b; d = b+c;` — C depends on B (longer serial chain)

There are **no cross-iteration dependencies**: each iteration writes to a distinct `out[i]` slot.

Run both benchmarks:

```
cd ooo_lab/Q2/tomasulo
make run-inorder && make run-ooo
```

	In-Order	OOO
diamond		
chain		

Question 3: The inner loop of each benchmark, as compiled by the toolchain, is shown below. Execution latencies are listed in the default configuration above.

bench_diamond inner loop (one iteration):

```
ld   a5, 0(a4)      ; load arr[i]
mul  a2, a5, a5     ; a2 = a5 * a5
add  a5, a5, 1      ; a5 = a5 + 1
add  a5, a5, a2     ; a5 = a5 + a2
sd   a5, -8(a3)     ; store out_diamond[i]
```

bench_chain inner loop (one iteration):

```
ld   a5, 0(a4)      ; load arr[i]
add  a5, a5, 1      ; a5 = a5 + 1
mul  a3, a5, a5     ; a3 = a5 * a5
add  a5, a3, a5     ; a5 = a3 + a5
sd   a5, -8(a2)     ; store out_chain[i]
```

For each benchmark, identify the data dependencies between instructions and determine the minimum number of cycles one iteration must take. Compare your theoretical minimum with the measured OOO

cycles-per-iteration, and explain what this reveals about how the OOO scheduler handles the two dependency structures.

Submit: completed table and written answer to Q3.

Step 3 — Memory-Level Parallelism

Background

`ooo_lab/Q3/memlat/` compares two access patterns with the same total data volume:

- `bench_parallel` — each iteration loads from **two independent arrays** (`arrA[i]` and `arrB[i]`)
- `bench_serial` — each iteration loads from **one array** (`arr[i]`)

Arrays are sized to exceed L1D but fit in L2 (arrA/arrB: 20 KB each; arr: 40 KB), guaranteeing `LAT_L2_HIT = 8` cycle latency on every access.

Run both benchmarks:

```
cd ooo_lab/Q3/memlat
make run-inorder && make run-ooo
```

Question 4: Change `LAT_L2_HIT` in `include/cpu/exec_cfg.h` to **20** and then to **40**. After each change run `make clean && make`, then re-run both benchmarks. Record the results below, then restore `LAT_L2_HIT` to **8** and run `make clean && make`.

LAT_L2_HIT	bench	In-Order CPI	OOO CPI	Speedup
8 (default)	parallel			
8 (default)	serial			
20	parallel			
20	serial			
40	parallel			
40	serial			

Submit: completed table.

Question 5: Based on the table above, does the OOO speedup over in-order keep growing as `LAT_L2_HIT` increases, or does it converge? Identify the hardware resource that sets the ceiling, state what value the speedup converges to, and explain why.

Submit: written answer to Q5.